

DTS103TC

Data Structure and Algorithms

Lab 5: Hash Table

Dr. Qi Chen, Dr. Di Zhang, Dr. Md Maruf Hasan,
Dr. Xuechen Liu, Dr. Guangyu Ren and Dr. Bin Chen
School of AI and Advanced Computing

Learning outcome

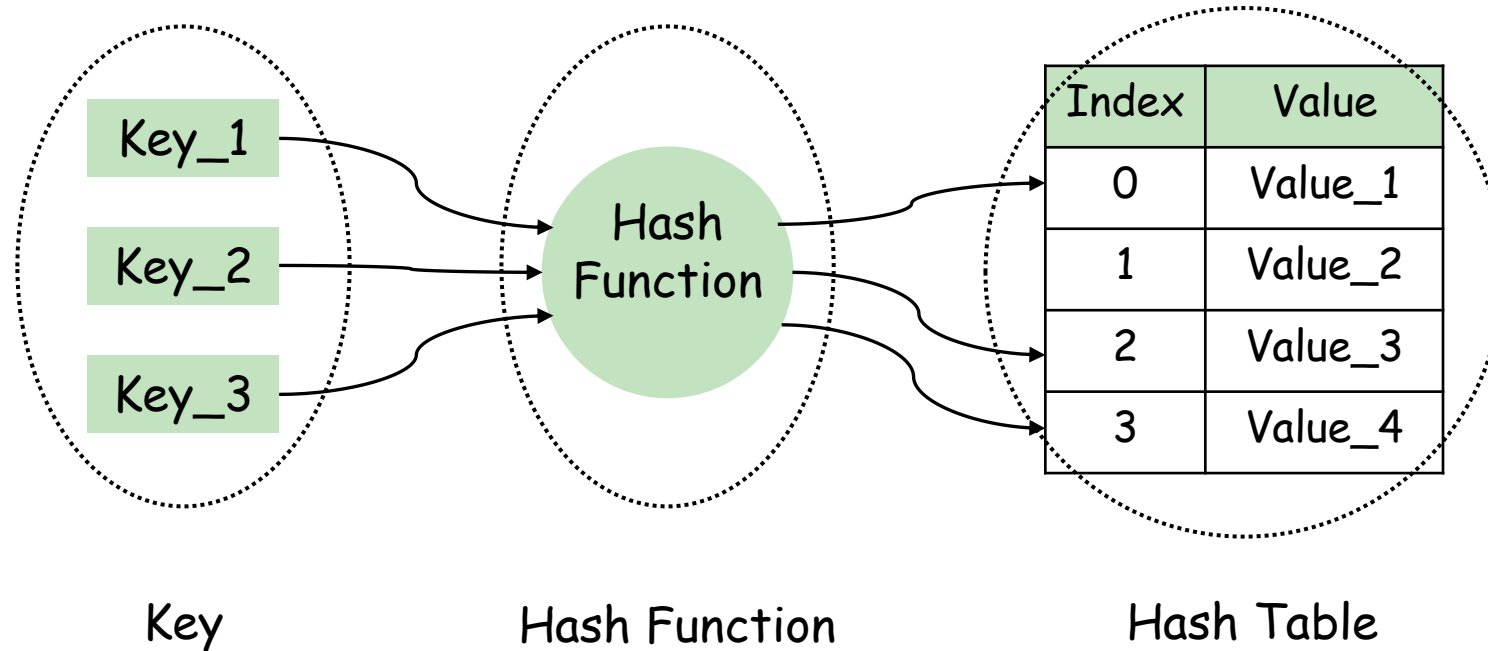
- Hash table
 - Hash function
 - Hash table implementation
 - Hash Table in Python (dict)
 - Exercises (Two Sum, frequency counting)

Hash Table

- Another kind of Table
- Use an array named T of capacity m
- Define a **hash function** that returns an integer $h(k)$
 - Must return an integer between 0 and $m-1$
 - $h()$ must always return the same integer for a given key
- Store the key and info at $T[h(k)]$
- $O(1)$ on **average** for insertion, lookup, and deletion

Hash Function

- Let k be a key and $h(k)$ be a hash function.
- The function $h(k)$ computes an index where the data associated with k will be stored.



Part 1: Python Built-in hash() Function

- `hash(x)` returns an integer hash value for an object
- To use in a hash table:
$$\text{index} = \text{hash}(\text{key}) \% m$$
- Properties:
 - Same input → same hash value (same within a run, may differ between runs)
 - Fast to compute $O(1)$
 - Works for many data types (int, str, tuple, etc.)

Part 1: Python Built-in hash() Function

Examples of hash() usage

```
print("Integer:", hash(10))  
print("String:", hash("hello"))  
print("Tuple:", hash((1, 2, 3)))
```

Try an unhashable object

```
try:  
    print(hash([1, 2, 3]))  
except TypeError as e:  
    print("Error:", e)
```

Integer: 10

String: 1762767184819215513

Tuple: 529344067295497451

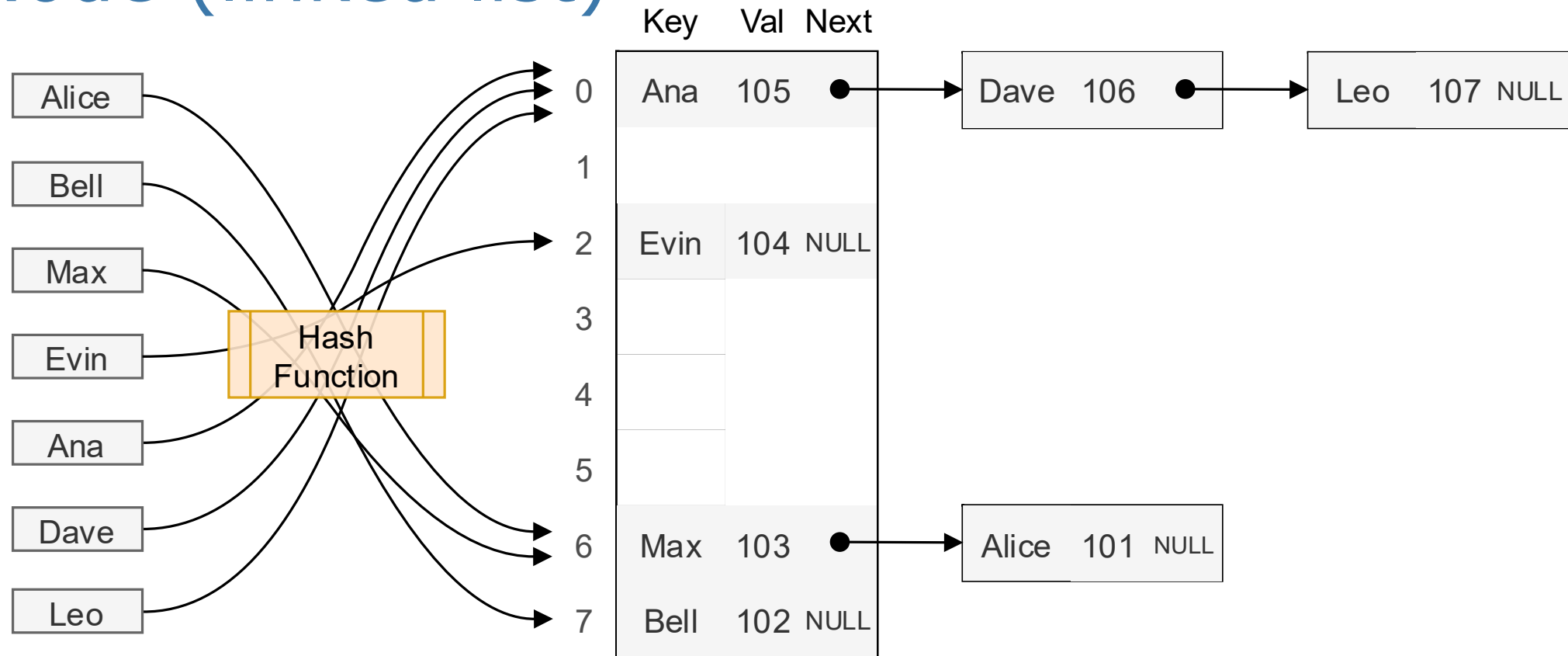
Error: unhashable type: 'list'

Part 2: Hash Table Implementation (Chaining)

- Components:
 - Node (linked list)
 - HashTable class
 - Hash function
 - Insert
 - Search
 - Remove

Hash Table Implementation (Chaining):

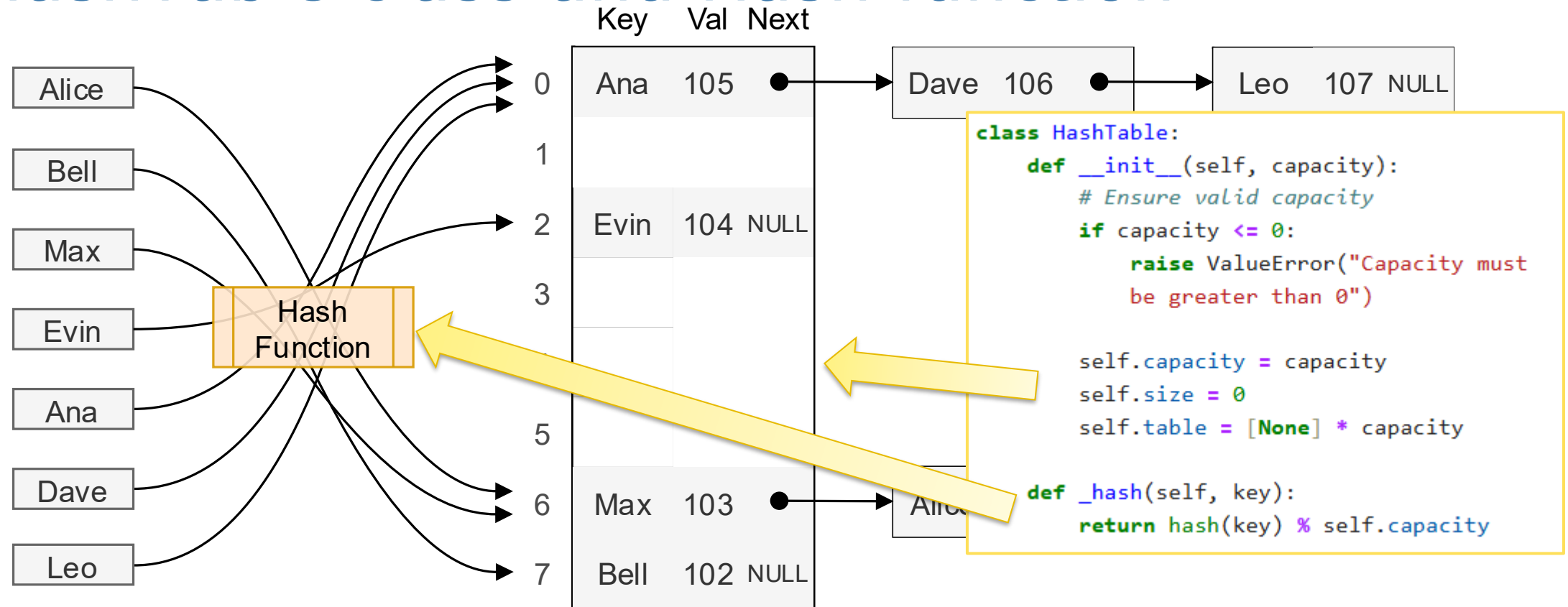
- Node (linked list)



```
class Node:
    def __init__(self, key, value):
        self.key = key          # key of the entry
        self.value = value      # value associated with the key
        self.next = None        # pointer to next node (for collision handling)
```

Hash Table Implementation (Chaining):

- HashTable class and Hash function



Hash Table Implementation (Chaining):

- Insert / Search / Remove

```
def insert(self, key, value):
    index = self._hash(key)

    # Case 1: No collision (bucket is empty)
    if self.table[index] is None:
        self.table[index] = Node(key, value)
        self.size += 1
        return

    # Case 2: Collision → traverse linked list
    current = self.table[index]
    while current.next:
        # If key already exists → update value
        if current.key == key:
            current.value = value
            return
        current = current.next

    # Check last node
    if current.key == key:
        current.value = value
    else:
        current.next = Node(key, value)
        self.size += 1
```

```
def search(self, key):
    index = self._hash(key)
    current = self.table[index]

    # Traverse linked list to find key
    while current:
        if current.key == key:
            return current.value
        current = current.next

    # Key not found
    raise KeyError(key)
```

```
def remove(self, key):
    index = self._hash(key)
    previous = None
    current = self.table[index]

    # Traverse linked list to find node to delete
    while current:
        if current.key == key:
            # Case 1: Node is at the head
            if previous is None:
                self.table[index] = current.next
            else:
                # Case 2: Node is in the middle or end
                previous.next = current.next

            self.size -= 1
            return

        previous = current
        current = current.next

    # Key not found
    raise KeyError(key)
```

Part 3: Hash Tables in Python (dict)

- Python provides a built-in hash table: dict (dictionary)
- Stores data as key-value pairs

```
# Create a dictionary
ht = {}

# Insert key-value pairs
ht["apple"] = 3
ht["banana"] = 2
ht["cherry"] = 5

# Search (lookup)
print(ht["banana"]) # 2
```

```
# Update value
ht["banana"] = 4
print(ht["banana"]) # 4

# Check existence
print("apple" in ht) # True
print("durian" in ht) # False

# Delete a key
del ht["apple"]

print(ht)
```

```
2
4
True
False
{'banana': 4, 'cherry': 5}
```

Hash Tables in Python (dict)

- How dict Works Internally
 - Uses hash() to compute index
 - Stores data in an array
 - Handles collisions automatically
 - Automatically resizes when needed

Hash Tables in Python (dict)

Operation	Complexity (Average Case)
Insert	$O(1)$
Lookup	$O(1)$
Delete	$O(1)$
Worst Case: $O(n)$ -> due to collisions	

```
List lookup time (100 runs): 0.134443 seconds
Dict lookup time (100 runs): 0.000006 seconds
```

Check Jupyter
Notebook file Part 3

Exercise 1: Two Sum Problem

- Problem: Given an array of integers and a target value, find two numbers such that they add up to the target. Return the indices of the two numbers.
- Example:
 - Input: `nums = [2, 7, 11, 15]`, `target = 9`
 - Output: `[0, 1]` (because $2 + 7 = 9$)
- Idea: Use a hash table to store seen numbers and check if the complement exists.
- Checking all pairs $O(n^2)$; Using hash table $O(n)$

Exercise 2: First Non-Repeating Character

- Problem: Given a string s , return the index of the first non-repeating character. If it does not exist, return -1.
- Example:
 - $s = \text{"hash table"} \rightarrow \text{output: } 2$
 - $s = \text{"data structure"} \rightarrow \text{output: } 0$
- Idea: Use a dictionary to count character frequencies, then find the first character that appears once.
- Without hashing: $O(n^2)$ (check each character); With hash table: $O(n)$